Day 67 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 10, Day 5 reference

Watch: Stanford CS224N (Winter 2019), Lecture 7: Vanishing Gradients, Fancy RNNs (Stanford Online) | MIT 6.S191 (2020): Recurrent Neural Networks (Alexander Amini)

`day67_rnn_lstm_gru_compared.py`, segment by segment, in the order `main()` uses things. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula; the green **SYSTEM DIAGRAM** and the graphs are the ones the script saved. A glossary is at the end.

MATH BOX LEGEND

Symbols: E = embedding table · H_t = the cell's hidden state at word t · K = number of tags (17) · mask = 1 on real words, 0 on padding · TV = total-variation distance between two probability distributions

Segment 0 — Setup

Imports, paths and the shared settings

CODE

```
import copy
import os
import time
from collections import Counter, defaultdict
import numpy as np
from threadpoolctl import threadpool_limits
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
np.random.seed(42)
HERE = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(HERE, "..", "week-09-sequence-data-rnns-i", "ud_english_ewt_pos")
PAD, UNK = 0, 1
CELLS = ["rnn", "lstm", "gru"]
NAMES = {"rnn": "RNN", "lstm": "LSTM", "gru": "GRU"}
COLORS = {"rnn": "#7f7f7f", "lstm": "#1f77b4", "gru": "#2ca02c"}
SEEDS = [0, 1, 2]
EPOCHS = 25
```

- `DATA_DIR = os.path.join(HERE, "..", ...)` — Day 62's dataset is read from the Week 9 folder, not copied: ".." means the parent folder.
- `CELLS = ["rnn", "lstm", "gru"]` — the three settings every comparison loops over.
- `COLORS = {...}` — one colour per cell, so every plot uses the same colours.
- `RNN, LSTM, GRU and data functions` — copied verbatim from Days 56-66 (not shown again here; see those days' guides).

Segment 1 — One tagger, three cells

Tagger.__init__ and run_cell

CODE

```
class Tagger:
    def __init__(self, vocab_size, n_tags, cell="rnn", embed_dim=32, hidden_dim=32, lr=1.0,
        clip=1.0, seed=0, gate_bias=1.0):
        rng = np.random.RandomState(seed)
        self.cell, self.n_tags, self.hidden_dim = cell, n_tags, hidden_dim
        self.lr, self.clip = lr, clip
        self.E = rng.randn(vocab_size, embed_dim) * 0.1
        if cell == "rnn":
            self.Wxh = rng.randn(embed_dim, hidden_dim) * np.sqrt(1.0 / embed_dim)
            self.Whh = rng.randn(hidden_dim, hidden_dim) * np.sqrt(1.0 / hidden_dim)
            self.bh = np.zeros(hidden_dim)
            self.cell_params = ["Wxh", "Whh", "bh"]
        elif cell == "lstm":
            self.W, self.b = init_lstm(embed_dim, hidden_dim, seed, forget_bias=gate_bias)
            self.cell_params = ["W", "b"]
        else:
            self.W, self.b = init_gru(embed_dim, hidden_dim, seed, keep_bias=gate_bias)
            self.cell_params = ["W", "b"]
        self.Why = rng.randn(hidden_dim, n_tags) * np.sqrt(1.0 / hidden_dim)
        self.by = np.zeros(n_tags)
        self.params = ["E"] + self.cell_params + ["Why", "by"]
    def n_weights(self):
        return {name: getattr(self, name).size for name in self.params}
    def run_cell(self, X):
        h0 = np.zeros((X.shape[0], self.hidden_dim))
        if self.cell == "rnn":
            return rnn_forward(X, h0, self.Wxh, self.Whh, self.bh), h0
        if self.cell == "lstm":
            Hs, _, self.cache = lstm_forward(X, h0, h0, self.W, self.b)
            return Hs, h0
```

- `self.E = rng.randn(...) * 0.1` — drawn FIRST from the seed, so all three cells start from the same word vectors.
- `if cell == "rnn": ... elif cell == "lstm": ... else:` — a branch picks how the cell's weights are created and named.
- `self.cell_params = ["W", "b"]` — the gated cells keep all their gates in one W and one b (Days 63 and 66).
- `self.params = ["E"] + self.cell_params + ["Why", "by"]` — a list of attribute NAMES: training and checking loop over it with getattr.
- `{name: getattr(self, name).size for name in self.params}` — a dictionary comprehension: parameter name -> number of entries.
- `Hs, _, self.cache = lstm_forward(X, h0, h0, ...)` — the LSTM also returns cell states; _ discards them. Both states start at zero.
- `return ..., h0` — the RNN's backward pass needs h0; the gated cells keep theirs in the cache.

forward, loss and gradients

MATH

$$probs = softmax(H_t W_{hy} + b_y) \text{ for every word } t$$

$$L = -\Sigma\, mask \cdot \log p(\text{true tag}) / \Sigma\, mask; \quad dlogits = (probs - onehot) \cdot mask / \Sigma\, mask$$

CODE

```
def forward(self, ids):
    m, T = ids.shape
    self.ids = ids
    self.X = self.E[ids]
    self.H, self.h0 = self.run_cell(self.X)
    self.H2 = self.H.reshape(m * T, self.hidden_dim)
    self.probs = softmax(self.H2 @ self.Why + self.by)
    return self.probs
def loss(self, tags, mask):
    p_true = self.probs[np.arange(tags.size), tags.ravel()]
    return -np.sum(np.log(p_true + 1e-9) * mask.ravel()) / mask.sum()
def gradients(self, tags, mask):
    m, T = self.ids.shape
    dlogits = (self.probs - one_hot(tags.ravel(), self.n_tags)) * mask.ravel()[:, None] /
        mask.sum()
    grads = dict(Why=self.H2.T @ dlogits, by=dlogits.sum(axis=0))
    dH = (dlogits @ self.Why.T).reshape(m, T, self.hidden_dim)
    zeros = np.zeros((m, self.hidden_dim))
    if self.cell == "rnn":
        grads["Wxh"], grads["Whh"], grads["bh"], dX = rnn_backward(dH, self.X, self.H, self.h0,
            self.Wxh, self.Whh)
    elif self.cell == "lstm":
        grads["W"], grads["b"], dX, _, _ = lstm_backward(dH, zeros, zeros, self.cache)
    else:
        grads["W"], grads["b"], dX, _ = gru_backward(dH, zeros, self.cache)
    grads["E"] = np.zeros_like(self.E)
    np.add.at(grads["E"], self.ids, dX)
    return grads
```

- `self.H2 = self.H.reshape(m * T, self.hidden_dim)` — every word of every sentence becomes one row, so one matrix product scores them all.
- `mask.ravel()[:, None] / mask.sum()` — padding rows get zero gradient; dividing by the real word count gives the mean loss.
- `dH = (dlogits @ self.Why.T).reshape(m, T, H)` — a gradient for every word's hidden state (Day 62's many-to-many line).
- `grads["W"], grads["b"], dX, _, _ = lstm_backward(dH, zeros, zeros, self.cache)` — five return values; the gradients at h0 and c0 are not needed.
- `np.add.at(grads["E"], self.ids, dX)` — a word that appears twice gets both gradients added (plain indexing would keep only one).

predict, drop_caches and snapshot

CODE

```
def predict(self, ids):
    pred = self.forward(ids).argmax(axis=1).reshape(ids.shape)
    self.drop_caches()
    return pred
def drop_caches(self):
    for name in ["ids", "X", "H", "h0", "H2", "probs", "cache"]:
        self.__dict__.pop(name, None)
def snapshot(self):
    self.drop_caches()
    return copy.deepcopy(self)
```

- `self.__dict__.pop(name, None)` — removes an attribute if it exists; None means "no error if it is missing".
- `copy.deepcopy(self)` — an independent copy of every array, so later training does not change the snapshot.
- `why drop first?` — the first draft deep-copied the per-batch caches too: memory reached 2.1 GB and training slowed. Dropping them first changed no result.

fit and fit_cached

CODE

```
def fit(data, vocab_size, n_tags, seed, epochs=EPOCHS, batch_size=32, **net_kwargs):
    (ids_tr, tags_tr, mask_tr), (ids_va, tags_va, mask_va), (ids_te, tags_te, mask_te) = data
    net = Tagger(vocab_size, n_tags, seed=seed, **net_kwargs)
    rng = np.random.RandomState(seed)
    lens_tr = mask_tr.sum(axis=1)
    hist = dict(loss=[], train=[], val=[], test=[], seconds=[])
    best_val, best_net, best_epoch = -1.0, None, -1
    for epoch in range(epochs):
        start_time = time.perf_counter()
        perm = rng.permutation(len(ids_tr))
        losses = []
        for start in range(0, len(perm), batch_size):
            idx = perm[start:start + batch_size]
            T_batch = lens_tr[idx].max()
            losses.append(net.train_step(ids_tr[idx, :T_batch], tags_tr[idx, :T_batch],
                mask_tr[idx, :T_batch]))
        hist["seconds"].append(time.perf_counter() - start_time)
        hist["loss"].append(np.mean(losses))
        hist["train"].append(token_accuracy(net.predict(ids_tr), tags_tr, mask_tr))
        hist["val"].append(token_accuracy(net.predict(ids_va), tags_va, mask_va))
        hist["test"].append(token_accuracy(net.predict(ids_te), tags_te, mask_te))
        if hist["val"][-1] > best_val:
            best_val, best_net, best_epoch = hist["val"][-1], net.snapshot(), epoch
    return best_net, best_epoch, {k: np.array(v) for k, v in hist.items()}
RUNS = {}
def fit_cached(data, vocab_size, n_tags, seed, **net_kwargs):
    key = (seed, tuple(sorted(net_kwargs.items())))
    if key not in RUNS:
        RUNS[key] = fit(data, vocab_size, n_tags, seed, **net_kwargs)
    return RUNS[key]
```

- `**net_kwargs` — collects any extra keyword arguments (cell=..., lr=...) and passes them on to Tagger.
- `T_batch = lens_tr[idx].max()` — pad each batch only to its own longest sentence.
- `time.perf_counter()` — a high-resolution clock; the difference of two readings is elapsed seconds.
- `key = (seed, tuple(sorted(net_kwargs.items())))` — dictionaries cannot be keys, so the settings become a sorted tuple.
- `RUNS[key]` — a run requested by two sections is trained once and reused.

Segment 2 — Checks and fair settings

The gradient check, for each cell

CODE

```
for cell in CELLS:
    net = Tagger(12, 3, cell=cell, embed_dim=4, hidden_dim=5, clip=None, seed=1)
    net.forward(ids)
    analytic = net.gradients(tags, mask)
    ok_all = True
    for name in net.params:
        numeric = numeric_gradient(net, name, ids, tags, mask)
        diff = np.abs(numeric - analytic[name])
        rel = diff / (np.abs(numeric) + np.abs(analytic[name]) + 1e-12)
        ok = bool(np.all(diff <= 1e-10 + 1e-6 * np.abs(numeric)))
        ok_all = ok_all and ok
        print(f"{NAMES[cell]:<6}{name:>10}{numeric.size:>9}{rel.max():>20.2e}{diff.max():>18.2e}{s
            tr(ok):>7}")
    verdict[cell] = ok_all and bool(np.all(analytic["E"][PAD] == 0))
```

- `for name in net.params:` — every parameter of THIS cell's tagger: 6 arrays for the RNN, 5 for the gated cells.
- `diff <= 1e-10 + 1e-6 * np.abs(numeric)` — Day 62's pass rule: a tiny absolute allowance plus a relative one.
- `analytic["E"][PAD] == 0` — padding's embedding row must receive no gradient at all.

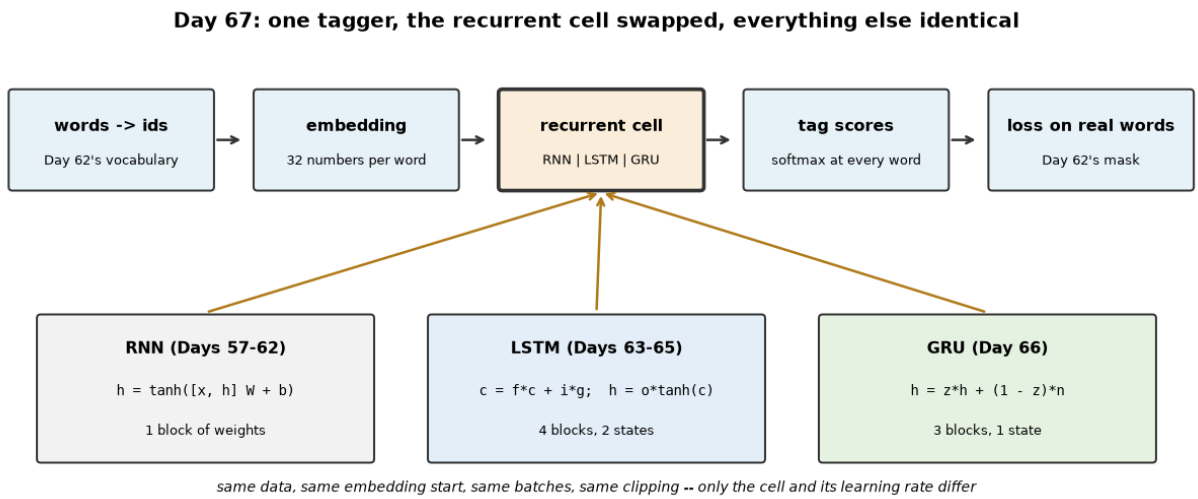
Choosing each cell's settings on validation

CODE

```
lrs = [0.5, 1.0, 2.0, 4.0]
grid = {"rnn": [(lr, 0.0) for lr in lrs],
        "lstm": [(lr, gb) for gb in [0.0, 1.0] for lr in lrs],
        "gru": [(lr, gb) for gb in [0.0, 1.0] for lr in lrs]}
print(f"\n{'cell':<6}{'lr':>5}{'gate bias':>11}{'best val':>10}{'at epoch':>10}{'val at
    end':>12}{'train at end':>14}")
chosen = {}
for cell in CELLS:
    best = (-1.0, None)
    for lr, gb in grid[cell]:
        _, be, h = fit_cached(data, V, K, seed=0, cell=cell, lr=lr, gate_bias=gb)
        print(f"{NAMES[cell]:<6}{lr:>5}{gb if cell != 'rnn' else
            '-':>11}{h['val'][be]:>10.4f}{be:>10}"
              f"{h['val'][-1]:>12.4f}{h['train'][-1]:>14.4f}")
        if h["val"][be] > best[0]:
            best = (h["val"][be], (lr, gb))
    chosen[cell] = best[1]
```

- `[(lr, gb) for gb in [0.0, 1.0] for lr in lrs]` — a list comprehension with two loops: every (learning rate, gate bias) pair.
- `h["val"][be]` — validation accuracy at the run's best epoch; the test column is never read here.
- `gb if cell != 'rnn' else '-'` — a conditional expression: the RNN has no gate bias, so a dash is printed.

SYSTEM DIAGRAM



One tagger with a slot for the cell; the three cells that fit it.

Segment 3 — Where the cells differ

Word groups and position groups

CODE

```
pos = np.tile(np.arange(ids_te.shape[1]), (len(ids_te), 1))
spans = [(0, 1), (1, 3), (3, 10), (10, 20), (20, 100)]
span_names = ["1st word", "2nd-3rd", "4th-10th", "11th-20th", "21st on"]
for (a, b), name in zip(spans, span_names):
    groups[name] = mask_te & (pos >= a) & (pos < b)
```

- `np.tile(np.arange(T), (n, 1))` — repeat the row 0, 1, ..., T-1 once per sentence: each word's position.
- `zip(spans, span_names)` — walk two lists side by side.
- `mask_te & (pos >= a) & (pos < b)` — boolean arrays combined with &: real words whose position is in [a, b).

The look-back probe

MATH

$$TV(p, q) = \tfrac{1}{2} \Sigma_k \, | \, p_k - q_k \, | \quad (0 = \text{identical}, 1 = \text{no overlap})$$

CODE

```
rng = np.random.RandomState(100 + k)
rows = np.where(lens > k)[0]
draws = []
for _ in range(5):
    for r in rows:
        t = rng.randint(k, lens[r])
        draws.append((r, t, rng.randint(2, V)))
draws = np.array(draws)
counts.append(len(draws))
r_idx, t_idx, new_w = draws[:, 0], draws[:, 1], draws[:, 2]
changed = ids_te[r_idx].copy()
changed[np.arange(len(draws)), t_idx - k] = new_w
same = changed[np.arange(len(draws)), t_idx - k] == ids_te[r_idx, t_idx - k]
for cell in CELLS:
    flips, tvs = [], []
    for s in SEEDS:
        net = fit_cached(data, V, K, seed=s, cell=cell, lr=chosen[cell][0],
            gate_bias=chosen[cell][1])[0]
        p_orig = probs_at(net, ids_te[r_idx], t_idx)
        p_new = probs_at(net, changed, t_idx)
        keep = ~same
        flips.append((p_orig.argmax(1) != p_new.argmax(1))[keep].mean())
        tvs.append(0.5 * np.abs(p_orig - p_new).sum(axis=1)[keep].mean())
    results[cell]["flip"].append(np.mean(flips))
    results[cell]["tv"].append(np.mean(tvs))
```

- `rng = np.random.RandomState(100 + k)` — the same random changes for every cell, so the cells are compared on identical edits.
- `changed[np.arange(len(draws)), t_idx - k] = new_w` — fancy indexing: in row j, set position t_idx[j] - k.
- `keep = ~same` — ~ flips a boolean array: keep only draws where the word really changed.
- `0.5 * np.abs(p_orig - p_new).sum(axis=1)` — total-variation distance between two distributions, one value per draw.

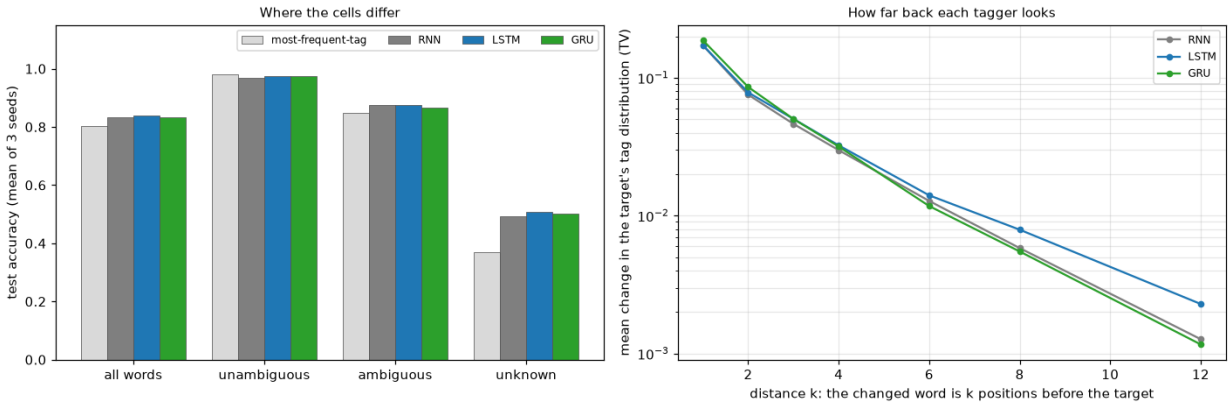
probs_at: the forward pass in chunks

CODE

```
def probs_at(net, ids, t_idx, chunk=1000):
    out = []
    for s in range(0, len(ids), chunk):
        p = net.forward(ids[s:s + chunk]).reshape(len(ids[s:s + chunk]), ids.shape[1], -1)
        out.append(p[np.arange(len(p)), t_idx[s:s + chunk]])
    net.drop_caches()
    return np.concatenate(out)
```

- `for s in range(0, len(ids), chunk):` — 8,000+ edited sentences at once would need over a gigabyte of gate arrays; 1,000 at a time does not.
- `p[np.arange(len(p)), t_idx[s:s + chunk]]` — for each sentence, keep only the row of its target word.
- `np.concatenate(out)` — join the chunks back into one array.

REAL GRAPH



Real graph from the script: accuracy by word group, and the look-back probe.

Segment 4 — Cost

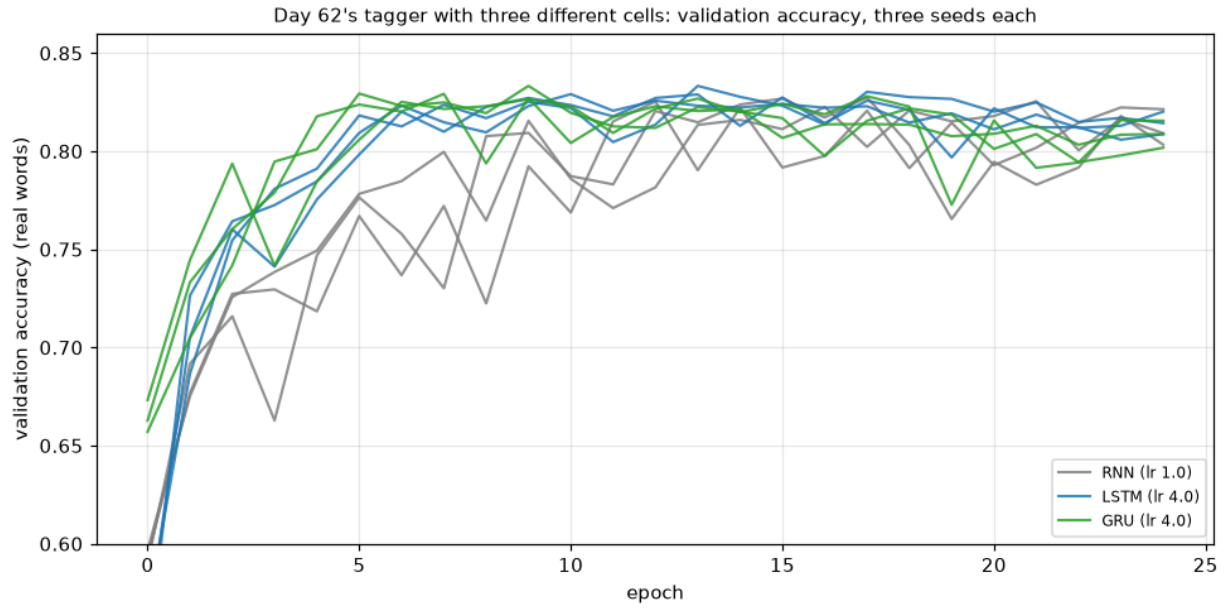
Timing from the training record

CODE

```
secs = np.concatenate([fit_cached(data, V, K, seed=s, cell=cell, lr=chosen[cell][0],
    gate_bias=chosen[cell][1])[2]["seconds"]
                       for s in SEEDS])
med[cell] = np.median(secs)
```

- `np.concatenate([...for s in SEEDS])` — join the per-epoch timings of the three seeds into one array.
- `np.median(secs)` — the middle value: one slow epoch (the computer busy elsewhere) does not move it.

REAL GRAPH



Real graph from the script: validation accuracy per epoch, three seeds per cell.

Quick Syntax Glossary — new or reinforced constructs

- `getattr(obj, name)` — an attribute chosen by a string; lets one loop handle every parameter.
- `self.__dict__.pop(name, None)` — delete an attribute if present, silently if not.
- `copy.deepcopy(x)` — a fully independent copy, arrays included.
- `**kwargs` — pass extra keyword arguments through to another function.
- `tuple(sorted(d.items()))` — turn a dictionary into something usable as a dictionary key.
- `np.tile(row, (n, 1))` — stack n copies of a row.
- `np.median(a)` — the middle value; robust to a few outliers.
- `a if cond else b` — an expression that picks one of two values.

End of Day 67 syntax reference. For the experiments and what they showed, see the Day 67 complete guide.